(https://hackmd.io/@mprocess / Lab2: Addressing Mode



# Lab2: Addressing Mode

## No Operation

- NOP
  - PC += 2
  - "wasting" 1 clock cycle
  - delay loop

| NOP | No Operation | | | |
|---|---|---|---|---|
| **Syntax** | **NOP** | | | |
| Operands | None | | | |
| Operation | No operation | | | |
| Status Affected | None | | | |
| Encoding | 0000 | 0000 | 0000 | 0000 |
| | 1111 | xxxx | xxxx | xxxx |
| Description | No operation | | | |
| Words | 1 | | | |
| Cycles | 1 | | | |

Q Cycle Activity:

| Q1 | Q2 | Q3 | Q4 |
|---|---|---|---|
| Decode | No operation | No operation | No operation |

Example: None.

Sample code:

```
#INCLUDE <p18f4520.inc>
        CONFIG OSC = INTIO67
        CONFIG WDT = OFF
        org 0x10 ; PC = 0x10
start:
        nop
        nop
        nop
        nop
        nop
end
```

# Inherent Addressing

- SLEEP

| SLEEP | Enter Sleep Mode |
|---|---|
| **Syntax** | **SLEEP** |
| Operands | None |
| Operation | 00h → WDT<br>1 → $\overline{TO}$<br>0 → PD |
| Status Affected | $\overline{TO}$, $\overline{PD}$ |
| Encoding | 0000 0000 0000 0011 |
| Description | The Power-down Status ($\overline{PD}$) bit is cleared. The Time-Out Status $\overline{TO}$) bit is set. Watchdog Timer is cleared. The processor is put into Sleep mode with the oscillator stopped. |
| Words | 1 |
| Cycles | 1 |

Q Cycle Activity:

| Q1 | Q2 | Q3 | Q4 |
|---|---|---|---|
| Decode | No operation | Process Data | Go to Sleep |

Example: SLEEP

Before Instruction
$\overline{TO}$ = ?
PD = ?

After Instruction
$\overline{TO}$ = 1 †
PD = 0

† If WDT causes wake-up, this bit is cleared.

- RESET

| RESET | Reset |
|---|---|
| **Syntax** | **RESET** |
| Operands | None |
| Operation | Reset all registers and flags that are affected by a $\overline{\text{MCLR}}$ Reset |
| Status Affected | All |
| Encoding | `0000 0000 1111 1111` |
| Description | This instruction provides a way to execute a $\overline{\text{MCLR}}$ Reset by software |
| Words | 1 |
| Cycles | 1 |

Q Cycle Activity:

| Q1 | Q2 | Q3 | Q4 |
|---|---|---|---|
| Decode | Start Reset | No operation | No operation |

Example: RESET

Before Instruction
All Registers = ?
All Flags = ?

After Instruction
All Registers = Reset Value
All Flags = Reset Value

- DAW

| DAW | Decimal Adjust W Register |
|---|---|
| **Syntax** | `DAW` |
| Operands | None |
| Operation | If [(W<3:0>) > 9] or [DC = 1] then<br>(W<3:0>) + 6 → W<3:0>;<br>else<br>(W<3:0>) → W<3:0>;<br>If [(W<7:4>) + DC > 9] or [C = 1] then<br>(W<7:4>) + 6 + DC → W<7:4>;<br>else<br>(W<7:4>) + DC → W<7:4> |
| Status Affected | C |
| Encoding | `0000 0000 0000 0111` |
| Description | `DAW` adjusts the 8-bit value in W, resulting from the earlier addition of two variables (each in packed BCD format) and produces a correct packed BCD result. |
| Words | 1 |
| Cycles | 1 |

Q Cycle Activity:

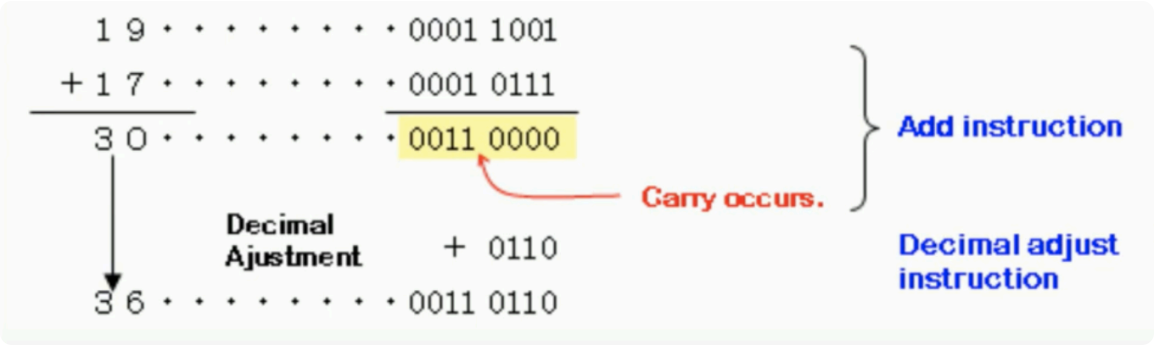
| Q1 | Q2 | Q3 | Q4 |
|---|---|---|---|
| Decode | Read register W | Process Data | Write register W |

Example 1: `DAW`

Before Instruction
W = `A5h`
C = `0`
DC = `0`

After Instruction
W = `05h`
C = `1`
DC = `0`

- BCD addition adjustment




## Literal Addressing

- 8-bits literal `MOVLW`、`ADDLW`、`SUBLW`、`ANDLW` ...
- 20-bits literal `GOTO`、`CALL` ...

Sample code:

```
#INCLUDE <p18f4520.inc>
        CONFIG OSC = INTIO67
        CONFIG WDT = OFF
        org 0x10 ; PC = 0x10
start:
    MOVLW 0x05

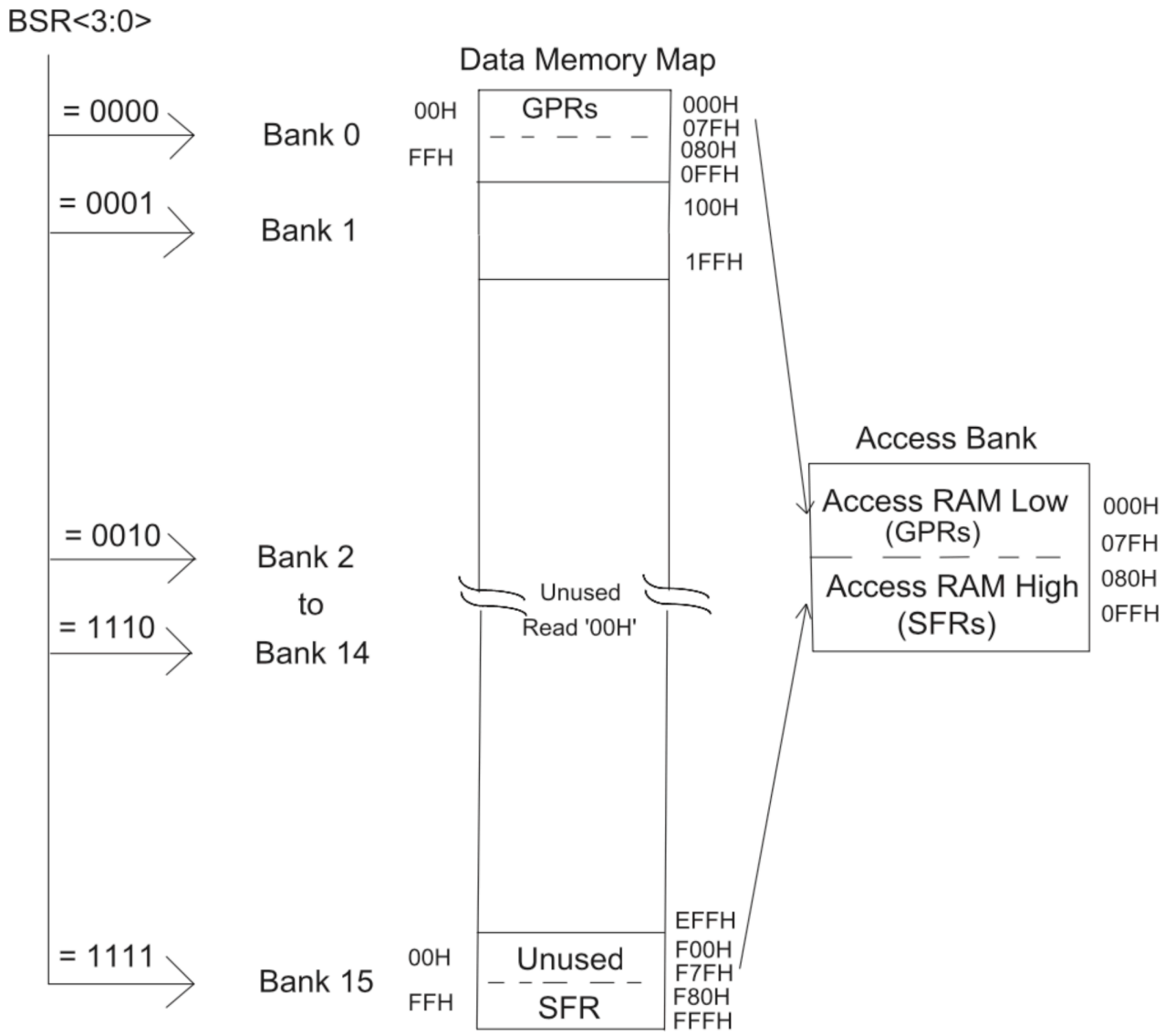
end

```

| Address ▲ | Name | Hex | Decimal | Binary | Char |
|---|---|---|---|---|---|
| FE2 | FSR1H | 0x00 | 0 | 00000000 | '.' |
| FE3 | PLUSW1 | 0x00 | 0 | 00000000 | '.' |
| FE4 | PREINC1 | 0x00 | 0 | 00000000 | '.' |
| FE5 | POSTDEC1 | 0x00 | 0 | 00000000 | '.' |
| FE6 | POSTINC1 | 0x00 | 0 | 00000000 | '.' |
| FE7 | INDF1 | 0x00 | 0 | 00000000 | '.' |
| FE8 | WREG | 0x05 | 5 | 00000101 | '.' |
| FE9 | FSR0 | 0x0000 | 0 | 00000000 00000000 | '..' |
| FE9 | FSR0L | 0x00 | 0 | 00000000 | '.' |
| FEA | FSR0H | 0x00 | 0 | 00000000 | '.' |
| FEB | PLUSW0 | 0x00 | 0 | 00000000 | '.' |
| FEC | PREINC0 | 0x00 | 0 | 00000000 | '.' |
| FED | POSTDEC0 | 0x00 | 0 | 00000000 | '.' |
| FEE | POSTINC0 | 0x00 | 0 | 00000000 | '.' |
| FEF | INDF0 | 0x00 | 0 | 00000000 | '.' |
| FF0 | INTCON3 | 0xC0 | 192 | 11000000 | 'À' |
| FF1 | INTCON2 | 0xF5 | 245 | 11110101 | 'õ' |
| FF2 | INTCON | 0x01 | 1 | 00000001 | '.' |

# Direct Addressing

Data Memory Map




**12-bit memory address, higher 4 bits for bank select.**

**→ 將整塊 4096 Bytes 的記憶體區分成 16 個小區（bank0 ~ bank15）**

> **Some data movement instructions on file register**
>
> 1. Access Bank
>
>  Access RAM (or GPRs): `0x000 ~ 0x07F` and `0xF80 ~ 0xFFF`
> 2. Bank Select (higher address): `0x000 ~ 0xFFF`

- [ ] 因為 data movement 的 file register 欄位只有 8 bits，所以 Access Bank 方式只能存取 256 bytes 的記憶體空間（ `0x000 ~ 0x07F` 和 `0xF80 ~ 0xFFF` ）。
  因此若要存取整個 4096 bytes 大小的記憶體空間，要使用 Bank Select 的方式存取。

- [ ] MOVWF

| MOVWF | Move W to f |
|---|---|
| Syntax | `MOVWF f {,a}` |
| Operands | 0 ≤ f ≤ 255<br>a ∈ [0, 1] |
| Operation | (W) → f |
| Status Affected | None |
| Encoding | `0110 111a ffff ffff` |
| Description | Move data from W to register 'f'. Location 'f' can be anywhere in the 256-byte bank.<br>If 'a' is '0', the Access Bank is selected. If 'a' is '1', the BSR is used to select the GPR bank.<br>If 'a' is '0' and the extended instruction set is enabled, this instruction operates in Indexed Literal Offset Addressing mode whenever f ≤ 95 (5Fh). See Byte-Oriented and Bit-Oriented Instructions in Indexed Literal Offset Mode for details. |
| Words | 1 |
| Cycles | 1 |

Q Cycle Activity:

| Q1 | Q2 | Q3 | Q4 |
|---|---|---|---|
| Decode | Read W | Process Data | Write register 'f' |

Example: `MOVWF REG, 0`

Before Instruction
W = 4Fh
REG = FFh

After Instruction
W = 4Fh
REG = 4Fh

- [ ] MOVLB (Use MOVLB to select bank)

| MOVLB | Move Literal to BSR |
|---|---|
| **Syntax** | `MOVLB k` |
| Operands | $0 \le k \le 63$ |
| Operation | $k \rightarrow BSR$ |
| Status Affected | None |
| Encoding | `0000 0001 00kk kkkk` |
| Description | The 6-bit literal 'k' is loaded into the Bank Select Register (BSR<5:0>). The value of BSR<7:6> always remains '0'. |
| Words | 1 |
| Cycles | 1 |

Q Cycle Activity:

| Q1 | Q2 | Q3 | Q4 |
|---|---|---|---|
| Decode | Read literal 'k' | Process Data | Write to BSR |

Example: `MOVLB 5`

Before Instruction
BSR = `02h`

After Instruction
BSR = `05h`

- MOVFF

| MOVFF | Move f to f | | | |
|---|---|---|---|---|
| Syntax | `MOVFF` $f_s$, $f_d$ | | | |
| Operands | $0 \le f_s \le 4095$<br>$0 \le f_d \le 4095$ | | | |
| Operation | $(f_s) \rightarrow f_d$ | | | |
| Status Affected | None | | | |
| Encoding | 1100 | $f_sf_sf_sf_s$ | $f_sf_sf_sf_s$ | $f_sf_sf_sf_s$ |
| | 1111 | $f_df_df_df_d$ | $f_df_df_df_d$ | $f_df_df_df_d$ |
| Description | The contents of source register '$f_s$' are moved to destination register '$f_d$'. Location of source '$f_s$' can be anywhere in the 4096-byte data space (`000h` to `FFFh`) and location of destination '$f_d$' can also be anywhere from `000h` to `FFFh`.<br>`MOVFF` is particularly useful for transferring a data memory location to a peripheral register (such as the transmit buffer or an I/O port).<br>The `MOVFF` instruction cannot use the PCL, TOSU, TOSH or TOSL as the destination register.<br>**Note:**<br>`MOVFF` has curtailed the source and destination range to the lower 4 Kbyte space of memory (Banks 1 through 15). For everything else, use `MOVFFL`. | | | |
| Words | 2 | | | |
| Cycles | 2 | | | |

Q Cycle Activity:

| Q1 | Q2 | Q3 | Q4 |
|---|---|---|---|
| Decode | Read register '$f_s$' | Process Data | No operation |
| Decode | No operation<br>No dummy read | No operation | Write register '$f_d$' |

`MOVFF` 是一個很特別的指令，它的指令格式是給你兩個 12-bit file register 欄位去填，所以在整個 4096 bytes 大小的記憶體裡，想去哪就去哪，無須在乎 bank；也就是說這個指令可以隨意在 4096 bytes 大小的記憶體空間裡存取。

> Sample code: `Access Bank`

```
#INCLUDE <p18f4520.inc>
    CONFIG OSC = INTIO67
    CONFIG WDT = OFF
    org 0x10    ;PC = 0x10
start:
    MOVLW 0x99    ; WREG = 0x99
    MOVWF 0x10    ; [0x010] = 0x99
end
```

| Address | 00 | 01 | 02 | 03 | 04 | 05 | 06 | 07 | 08 | 09 | 0A | 0B | 0C | 0D | 0E | 0F | ASCII |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 000 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 010 | 99 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 020 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 030 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 040 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 050 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 060 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 070 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 080 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 090 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 0A0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 0B0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 0C0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 0D0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 0E0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 0F0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 100 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 110 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 120 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |

> Sample code: `Bank Select`

```
#INCLUDE <p18f4520.inc>
    CONFIG OSC = INTIO67
    CONFIG WDT = OFF
    org 0x10    ;PC = 0x10
start:
    MOVLW 0x99    ; WREG = 0x99
    MOVLB 0x4     ; BSR = 4
    MOVWF 0x10, 1 ; use BSR select bank ; [0x410] = 0x99
end
```

| Address ▲ | Name | Hex | Decimal | Binary | Char |
|---|---|---|---|---|---|
| FD9 | FSR2 | 0x0000 | 0 | 00000000 00000000 | '..' |
| FD9 | FSR2L | 0x00 | 0 | 00000000 | '.' |
| FDA | FSR2H | 0x00 | 0 | 00000000 | '.' |
| FDB | PLUSW2 | 0x00 | 0 | 00000000 | '.' |
| FDC | PREINC2 | 0x00 | 0 | 00000000 | '.' |
| FDD | POSTDEC2 | 0x00 | 0 | 00000000 | '.' |
| FDE | POSTINC2 | 0x00 | 0 | 00000000 | '.' |
| FDF | INDF2 | 0x00 | 0 | 00000000 | '.' |
| FE0 | BSR | 0x04 | 4 | 00000100 | '.' |
| FE1 | FSR1 | 0x0000 | 0 | 00000000 00000000 | '..' |
| FE1 | FSR1L | 0x00 | 0 | 00000000 | '.' |
| FE2 | FSR1H | 0x00 | 0 | 00000000 | '.' |
| FE3 | PLUSW1 | 0x00 | 0 | 00000000 | '.' |
| FE4 | PREINC1 | 0x00 | 0 | 00000000 | '.' |
| FE5 | POSTDEC1 | 0x00 | 0 | 00000000 | '.' |
| FE6 | POSTINC1 | 0x00 | 0 | 00000000 | '.' |
| FE7 | INDF1 | 0x00 | 0 | 00000000 | '.' |
| FE8 | WREG | 0x99 | 153 | 10011001 | ' ' |
| FE9 | FSR0 | 0x0000 | 0 | 00000000 00000000 | '..' |

| Address | 00 | 01 | 02 | 03 | 04 | 05 | 06 | 07 | 08 | 09 | 0A | 0B | 0C | 0D | 0E | 0F | ASCII |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 400 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 410 | 99 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 420 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 430 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 440 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 450 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 460 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 470 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 480 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 490 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 4A0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 4B0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 4C0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 4D0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 4E0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 4F0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 500 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 510 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 520 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |

> Sample code: `MOVFF`

```
#INCLUDE <p18f4520.inc>
    CONFIG OSC = INTIO67
    CONFIG WDT = OFF
    org 0x10    ;PC = 0x10
start:
    MOVLW 0x99            ; WREG = 0x99
    MOVLB 0x4             ; BSR = 4
    MOVWF 0x10, 1         ; use BSR select bank ; [0x410] = 0x99
    MOVFF 0x410, 0x420    ; [0x420] = 0x99
end
```

| Address | 00 | 01 | 02 | 03 | 04 | 05 | 06 | 07 | 08 | 09 | 0A | 0B | 0C | 0D | 0E | 0F | ASCII |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 400 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 410 | 99 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 420 | 99 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 430 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 440 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 450 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 460 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 470 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 480 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 490 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 4A0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 4B0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 4C0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 4D0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 4E0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 4F0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 500 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 510 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 520 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |

# Indirect Addressing

### Three SFRs call FSRx , x ∈ [0,2]

- FSR0、FSR1、FSR2
- 16bits ( `FSRxH` : `FSRxL` ) to cover all data memory address
- they are pointer

- LFSR (let FSRx point to memory address k)

| LFSR | Load FSR | | | |
|---|---|---|---|---|
| **Syntax** | **LFSR $f_n$, k** | | | |
| Operands | $0 \le f_n \le 2$<br>$0 \le k \le 16383$ | | | |
| Operation | $k \rightarrow FSRf_n$ | | | |
| Status Affected | None | | | |
| Encoding | 1110 | 1110 | $00f_nf_n$ | $k_{13}kkk_{10}$ |
| | 1111 | $00k_9k$ | kkkk | $kkkk_0$ |
| Description | The 14-bit literal 'k' is loaded into the File Select Register '$f_n$' | | | |
| Words | 2 | | | |
| Cycles | 2 | | | |

Q Cycle Activity:

| Q1 | Q2 | Q3 | Q4 |
|---|---|---|---|
| Decode | Read literal 'k'<13:10> | Process Data | Write literal 'k'<13:10> to $FSRf_n$<13:10> |
| No operation | Read literal 'k'<9:0> | No operation | Write literal 'k'<9:0> to $FSRf_n$<9:0> |

Example: `LFSR 2, 3ABh`

Before Instruction
FSR2H = ?
FSR2L = ?

After Instruction
FSR2H = `03h`
FSR2L = `ABh`

**Some SFRs related to pointer FSRx , x ∈ [0,2]**

- `INDFx` ：指針不變，對指向的記憶體位置進行操作
- `POSTINCx` ：對指向的記憶體位置進行操作後，指針 + 1
- `POSTDECx` ：對指向的記憶體位置進行操作後，指針 - 1
- `PREINCx` ：指針 + 1 後，對指向的記憶體位置進行操作
- `PLUSWx` ：指針 + WREG = 新的記憶體位置後，對指向的記憶體位置進行操作

> Sample code:

```
#INCLUDE <p18f4520.inc>
    CONFIG OSC = INTIO67
    CONFIG WDT = OFF
    org 0x00    ;PC = 0x00

setup1:
    LFSR 0, 0x000    ; FSR0 point to 0x000
    LFSR 1, 0x010    ; FSR1 point to 0x010
    LFSR 2, 0x020    ; FSR2 point to 0x020
    MOVLW 0x10       ; WREG = 0x10

start:
    INCF POSTINC0
    ; [0x000] += 1; FSR0 point to 0x001

    INCF PREINC1
    ; FSR1 point to 0x011 ;[0x011] += 1

    INCF POSTDEC2
    ; [0x020] += 1 ; FSR2 point to 0x01F

    INCF INDF2
    ; [0x01F] += 1 ;
    ; FSR2 point to 0x01F(unchanged)

    INCF PLUSW2
    ; [0x01F+0x10] += 1
    ; FSR2 point to 0x01F(unchanged)
end
```

# Bit Addressing

### BSF、BCF、BTFSC、BTFSS

- Set / Clear specific bit file register

- [ ] 
  - `BSF` ：Bit set f
  - `BCF` ：Bit clear f
  - `BTFSC` ：Bit test f skip if clear
  - `BTFSS` ：Bit test f skip if set

> Sample code:

```
#INCLUDE <p18f4520.inc>
    CONFIG OSC = INTIO67
    CONFIG WDT = OFF
    org 0x10    ;PC = 0x10
start:
    BSF   0x000, 1    ; [0x000] = b'00000010
    BTFSC 0x000, 0    ; test bit 0 of [0x000], skip if bit 0 is 0
    MOVFF 0x000, 0x001
    BTFSC 0x000, 1    ; test bit 1 of [0x000], skip if bit 1 is 0
    MOVFF 0x000, 0x001
end
```

| Address | 00 | 01 | 02 | 03 | 04 | 05 | 06 | 07 | 08 | 09 | 0A | 0B | 0C | 0D | 0E | 0F | ASCII |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 000 | 02 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 010 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 020 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 030 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 040 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 050 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 060 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 070 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 080 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 090 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 0A0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 0B0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 0C0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 0D0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 0E0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 0F0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 100 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 110 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 120 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |

| Address | 00 | 01 | 02 | 03 | 04 | 05 | 06 | 07 | 08 | 09 | 0A | 0B | 0C | 0D | 0E | 0F | ASCII |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 000 | 02 | 02 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 010 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 020 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 030 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 040 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 050 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 060 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 070 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 080 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 090 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 0A0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 0B0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 0C0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 0D0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 0E0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 0F0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 100 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 110 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |
| 120 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ........ ........ |

## Relative Addressing

- `BC ´ BN ´ BNC ´ BNN ´ BNZ ...`
- for all the "**Branch**" instruction to addressing
- relative address to PC value (branch 的下一行)
- offset is word address (for PIC18F4520, 1 word = 2 Bytes)

- [ ] if branch: $PC = PC + 2 + n * 2$ (2's complement)
  if not branch: $PC = PC + 2$

- BZ

| BZ | Branch if Zero |
|---|---|
| Syntax | `BZ n` |
| Operands | -128 ≤ n ≤ 127 |
| Operation | If ZERO bit is '1'<br>(PC) + 2 + 2n → PC |
| Status Affected | None |
| Encoding | `1110 0000 nnnn nnnn` |
| Description | If the ZERO bit is '1', then the program will branch. The two's complement number '2n' is added to the PC. Since the PC will have incremented to fetch the next instruction, the new address will be PC + 2 + 2n. This instruction is then a two-cycle instruction. |
| Words | 1 |
| Cycles | 1 (2) |

Q Cycle Activity:
If Jump:

| Q1 | Q2 | Q3 | Q4 |
|---|---|---|---|
| Decode | Read literal 'n' | Process Data | Write to PC |
| No operation | No operation | No operation | No operation |

If No Jump:

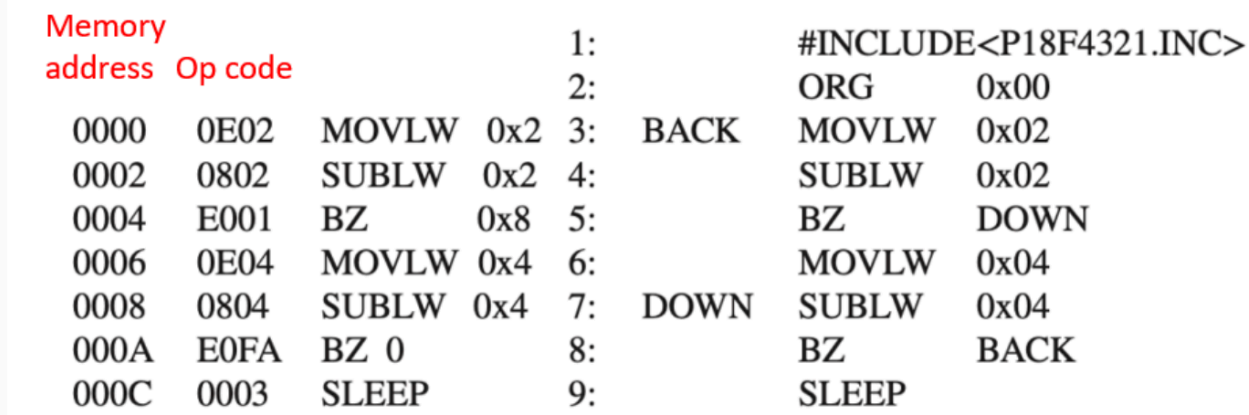
| Q1 | Q2 | Q3 | Q4 |
|---|---|---|---|
| Decode | Read literal 'n' | Process Data | No operation |

Example: `HERE BOV Jump`

Before Instruction
PC = address `(HERE)`

After Instruction
If ZERO = `1`; PC = address `(Jump)`
If ZERO = `0`; PC = address `(HERE + 2)`

- [ ] Example:



| Memory address | Op code | | | | | | |
|---|---|---|---|---|---|---|---|
| | | | | 1: | | #INCLUDE<P18F4321.INC> | |
| | | | | 2: | | ORG | 0x00 |
| 0000 | 0E02 | MOVLW | 0x2 | 3: | BACK | MOVLW | 0x02 |
| 0002 | 0802 | SUBLW | 0x2 | 4: | | SUBLW | 0x02 |
| 0004 | E001 | BZ | 0x8 | 5: | | BZ | DOWN |
| 0006 | 0E04 | MOVLW | 0x4 | 6: | | MOVLW | 0x04 |
| 0008 | 0804 | SUBLW | 0x4 | 7: | DOWN | SUBLW | 0x04 |
| 000A | E0FA | BZ | 0 | 8: | | BZ | BACK |
| 000C | 0003 | SLEEP | | 9: | | SLEEP | |

## Hard template

Please choose the version matching your assembler.
If you are using MPLAB X IDE v5.20, choose `MPASM` version; otherwise (MPLAB X IDE v6.xx, VSCode, etc.), choose the `PIC-AS` version

- MPASM:

```
LIST p=18f4520
#include "p18f4520.inc"

TESTCASE CODE
ORG 0x5000
; Sequence length, 0x02~0xff
DW  0x20
; The following is the input sequence.
; Here is an example sequence of length 64.
DB  0x88, 0x7b, 0xb6, 0xbd, 0xf5, 0xfa, 0x8c, 0x9a
DB  0xca, 0xec, 0x95, 0x69, 0x3f, 0x33, 0xae, 0x23
DB  0x3d, 0x29, 0x85, 0x80, 0x44, 0x8a, 0x99, 0xc9
DB  0x41, 0x09, 0xc1, 0x5c, 0x66, 0xba, 0xac, 0xa4
DB  0xe0, 0xcb, 0x64, 0xc6, 0xa1, 0xa5, 0x2f, 0xa8
DB  0x7c, 0xd3, 0x93, 0x0c, 0x07, 0xd1, 0xd2, 0xc6
DB  0x11, 0x34, 0x1f, 0x2b, 0x31, 0x72, 0x4e, 0x71
DB  0xc3, 0xb9, 0xf6, 0xf0, 0x74, 0x6c, 0x5f, 0x70


MAIN_PROG CODE
ORG 0x00
; Data loader, !DO NOT MODIFY!
DW 0x6af6, 0x6af8, 0x0e50, 0x6ef7, 0x0009, 0x50f5, 0x6f00, 0xee01
DW 0xf000, 0x000b, 0xcff5, 0xffee, 0x62e9, 0xd7fb, 0xee00, 0xf000

MAIN:
    ; your code here...

END
```

- PIC-AS

```
#include "xc.inc"
PSECT testcase, local, class=STRCODE, reloc=1
ORG 0x5000
; Sequence length, 0x02~0xff
DW  0x20
; The following is the input sequence.
; Here is an example sequence of length 64.
DB  0x88, 0x7b, 0xb6, 0xbd, 0xf5, 0xfa, 0x8c, 0x9a
DB  0xca, 0xec, 0x95, 0x69, 0x3f, 0x33, 0xae, 0x23
DB  0x3d, 0x29, 0x85, 0x80, 0x44, 0x8a, 0x99, 0xc9
DB  0x41, 0x09, 0xc1, 0x5c, 0x66, 0xba, 0xac, 0xa4
DB  0xe0, 0xcb, 0x64, 0xc6, 0xa1, 0xa5, 0x2f, 0xa8
DB  0x7c, 0xd3, 0x93, 0x0c, 0x07, 0xd1, 0xd2, 0xc6
DB  0x11, 0x34, 0x1f, 0x2b, 0x31, 0x72, 0x4e, 0x71
DB  0xc3, 0xb9, 0xf6, 0xf0, 0x74, 0x6c, 0x5f, 0x70

PSECT main, local, abs, class=CODE, reloc=2
ORG 0x00
; Data loader, !DO NOT MODIFY!
DW 0x6af6, 0x6af8, 0x0e50, 0x6ef7, 0x0009, 0x50f5, 0x6f00, 0xee01
DW 0xf000, 0x000b, 0xcff5, 0xffee, 0x62e9, 0xd7fb, 0xee00, 0xf000

main:
    ; your code here...
```

Answer of this example:

| Address | 0x00 | 0x01 | 0x02 | 0x03 | 0x04 | 0x05 | 0x06 | 0x07 |
|---|---|---|---|---|---|---|---|---|
| 0x100 | 0x09 | 0x23 | 0x29 | 0x33 | 0x3d | 0x3f | 0x41 | 0x44 |

| Address | 0x08 | 0x09 | 0x0A | 0x0B | 0x0C | 0x0D | 0x0E | 0x0F |
|---|---|---|---|---|---|---|---|---|
| 0x108 | 0x5c | 0x66 | 0x69 | 0x7b | 0x80 | 0x85 | 0x88 | 0x8a |

| Address | 0x00 | 0x01 | 0x02 | 0x03 | 0x04 | 0x05 | 0x06 | 0x07 |
|---|---|---|---|---|---|---|---|---|
| 0x110 | 0x8c | 0x95 | 0x99 | 0x9a | 0xa4 | 0xac | 0xae | 0xb6 |

- [ ]

| Address | 0x08 | 0x09 | 0x0A | 0x0B | 0x0C | 0x0D | 0x0E | 0x0F |
|---|---|---|---|---|---|---|---|---|
| 0x118 | 0xba | 0xbd | 0xc1 | 0xc9 | 0xca | 0xec | 0xf5 | 0xfa |

You can generate other testcases on https://mclab-2.chih-hao.xyz (https://mclab-2.chih-hao.xyz)